

Python

Summary Session 10



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что делает метод `fromkeys()`?



Метод `fromkeys()`

Этот метод позволяет создавать словарь, где указанным ключам присваивается одинаковое значение

Синтаксис



```
dict.fromkeys(iterable, value)
```

Пример создания словаря с неизменяемыми значениями



Создание словаря без указания значения

```
keys = ["x", "y", "z"]
```

```
my_dict = dict.fromkeys(keys) # Каждому ключу присваивается значение `None`
```

```
print(my_dict) # {'x': None, 'y': None, 'z': None}
```

Создание словаря с переданным значением

```
keys = [1, 2, 3]
```

```
default_value = "default"
```

```
my_dict = dict.fromkeys(keys, default_value)
```

```
print(my_dict) # {1: 'default', 2: 'default', 3: 'default'}
```


Экспресс-опрос

?

Что важно учитывать при использовании изменяемых значений в словарях?

Создание словаря с изменяемыми значениями



Будьте осторожны при использовании изменяемых значений (например, списка), так как все ключи будут ссылаться на один и тот же объект.

Пример создания словаря с изменяемыми значениями



```
keys = ["a", "b", "c"]  
  
shared_list = []  
  
my_dict = dict.fromkeys(keys, shared_list)  
  
my_dict["a"].append(1)  # Изменит общий список для всех ключей  
  
print(my_dict)  # {'a': [1], 'b': [1], 'c': [1]}
```

Экспресс-опрос

?

Что делает метод `get()`?



Метод `get()`

Этот метод используется для безопасного доступа к значению по указанному ключу.

Экспресс-опрос



Метод `get()` вызывает ошибку `KeyError`, если ключ отсутствует?

Метод get



В отличие от `dictionary[key]`, метод `get()` не вызывает ошибку `KeyError`, если ключ отсутствует, а вместо этого возвращает значение по умолчанию.

Синтаксис



```
value = dictionary.get(key, default_value)
```


Метод get



dictionary — словарь, из которого нужно получить значение.



key — ключ, по которому производится поиск значения.



default_value (необязательный параметр) — значение, которое будет возвращено, если ключ отсутствует. Если не указано, возвращается **None**.

Примеры



Получение значения по существующему ключу

```
my_dict = {"name": "Alice", "age": 30}
```

```
print(my_dict.get("name")) # Alice
```

```
print(my_dict.get("name", "Anonim")) # Alice
```

Запрос отсутствующего ключа без указания значения по умолчанию

```
print(my_dict.get("city")) # None
```

Запрос отсутствующего ключа с указанным значением по умолчанию

```
print(my_dict.get("city", "Unknown")) # Unknown
```

Экспресс-опрос

?

Что делает метод `setdefault()`?



Метод `setdefault()`

Этот метод используется для получения значения по указанному ключу. Если ключ отсутствует в словаре, метод добавляет его с указанным значением по умолчанию и возвращает это значение. Если ключ уже существует, `setdefault()` просто возвращает его текущее значение без изменений

Синтаксис



```
value = dictionary.setdefault(key, default_value)
```

Метод setdefault



dictionary — объект словаря, из которого вы хотите получить значение.



key — ключ, который нужно найти или добавить.



default_value (необязательный параметр) — значение, которое будет добавлено, если ключ отсутствует. Если не указано, используется **None**.

Примеры



Получение значения существующего ключа

```
my_dict = {"name": "Alice", "age": 30}
```

```
age = my_dict.setdefault("age")
```

```
print(age) # 30
```

```
print(my_dict) # {'name': 'Alice', 'age': 30}
```

Примеры



```
# Добавление нового ключа без указания значения по умолчанию
country = my_dict.setdefault("country")
print(country)  # None
print(my_dict)  # {'name': 'Alice', 'age': 30, 'country': None}
```


Примеры



```
# Добавление нового ключа с переданным значением

city = my_dict.setdefault("city", "Unknown")

print(city) # Unknown

print(my_dict) # {'name': 'Alice', 'age': 30, 'country': None, 'city': 'Unknown'}
```

Экспресс-опрос

?

Какие преимущества есть у метода `setdefault()`?

Преимущества метода `setdefault()`



Упрощает работу со словарями, когда необходимо гарантировать наличие ключа с определённым значением. Позволяет избежать дополнительных проверок `if key in dictionary` перед добавлением нового ключа.



ВОПРОСЫ



Экспресс-опрос

?

Словари в Python обладают тремя полезными методами для работы с ключами, значениями и парами "ключ-значение". Перечислите эти методы

Методы keys, values, items



Словари в Python обладают тремя полезными методами для работы с ключами, значениями и парами "ключ-значение". Эти методы возвращают динамические представления (*view objects*), которые ссылаются на словарь и автоматически отражают изменения в его содержимом.



Метод `keys()`

Этот метод возвращает представление всех ключей в словаре

Метод `keys()`



Представление обновляется автоматически при изменении словаря.



Можно преобразовать результат в список или другую коллекцию для получения копии элементов.

Пример



```
my_dict = {"name": "Alice", "age": 30}
keys = list(my_dict.keys())
print(type(keys))
print(keys)
```

```
# Изменение словаря
my_dict["city"] = "New York"
print(keys) # Значения обновляются
```

```
# Преобразование представления в список
keys_list = list(my_dict.keys())
my_dict["country"] = "USA"
print(keys_list) # На списке изменения не отображаются
```



Метод `values()`

Этот метод возвращает представление всех значений в словаре

Метод values()



Представление автоматически обновляется при изменении словаря.



Можно преобразовать результат в список или другую коллекцию для получения копии элементов.

Пример



```
my_dict = {"name": "Alice", "age": 30}
values = my_dict.values()
print(type(values))
print(values)
```

```
# Изменение словаря
my_dict["age"] = 31
print(values) # Значение обновляется
```

```
# Преобразование представления в список
values_list = list(values)
my_dict["age"] = 32
print(values_list) # Изменения на списке не отображаются
```



Метод items()

Этот метод возвращает представление всех пар "ключ-значение" в словаре в виде кортежей (ключ, значение)

Метод items()



Представление автоматически обновляется при изменении словаря.



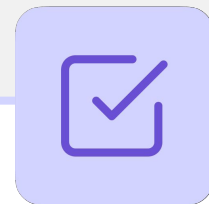
Можно преобразовать результат в список (с кортежами) или другую коллекцию для получения копии элементов.

Экспресс-опрос



Каким образом можно в программе перебирать все элементы словаря?

Цикл по словарю



Словари позволяют итерироваться по своим ключам, значениям или парам "ключ-значение" с помощью цикла `for`.

Итерация по ключам



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}
```

```
for key in my_dict:
```

```
    print(key)
```

```
# Аналогично циклу по my_dict
```

```
for key in my_dict.keys():
```

```
    print(key)
```

По умолчанию цикл for перебирает ключи словаря.

Экспресс-опрос

?

Какой метод используется для перебора по значениям?

Итерация по значениям



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}  
  
for value in my_dict.values():  
    print(value)
```

Для перебора значений используется метод `values()`.

Экспресс-опрос

?

Какой метод используется для перебора по парам ключ-значение?

Итерация по парам "ключ-значение"



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}  
for key, value in my_dict.items():  
    print(f"{key}: {value}")
```

Для перебора пар "ключ-значение" используется метод `items()`.



ВОПРОСЫ



Экспресс-опрос

?

Могут ли словари содержать вложенные структуры? Какие?

Словари со вложенными структурами



Словари могут содержать вложенные структуры данных, такие как списки, кортежи, множества и другие словари в качестве значений.

Списки внутри словаря



```
student_scores = {  
    "Alice": [90, 85, 88],  
    "Bob": [72, 75, 80],  
    "Charlie": [95, 100, 98]  
}
```

Вложенные словари и списки



```
school = {  
    "class1": {  
        "students": ["Alice", "Bob", "Charlie"],  
        "teacher": "Mrs. Smith"  
    },  
    "class2": {  
        "students": ["David", "Eva"],  
        "teacher": "Mr. Johnson"  
    }  
}
```

Доступ к элементам в словаре со вложенными структурами



Для доступа к элементам необходимо указывать ключи и индексы в зависимости от типа вложенной структуры.

Примеры



```
student_scores = {  
    "Alice": [90, 85, 88],  
    "Bob": [72, 75, 80],  
    "Charlie": [95, 100, 98]  
}
```

Доступ к элементу списка внутри словаря

```
print(student_scores["Alice"][1])
```

Примеры



```
school = {
    "class1": {
        "students": ["Alice", "Bob", "Charlie"],
        "teacher": "Mrs. Smith"
    },
    "class2": {
        "students": ["David", "Eva"],
        "teacher": "Mr. Johnson"
    }
}

# Доступ к элементу вложенного словаря
print(school["class2"]["teacher"])

# Доступ к элементу списка, полученного из вложенного словаря
print(school["class1"]["students"][0])
```

Итерация по словарю со вложенными структурами



В зависимости от типа вложенной структуры может потребоваться использовать вложенные циклы для перебора элементов.

Пример итерации



```
for class_name, details in school.items():  
    print(f"Class: {class_name}")  
    for key, value in details.items():  
        print(f"\t\t{key}: {value}")
```

Обновление элементов



Можно добавлять, изменять или удалять данные во вложенных структурах, используя доступ по ключам и индексам.

Экспресс-опрос

?

Какой метод используется для добавления элемента в словарь?

Пример добавления элемента



Добавление ученика в список

```
school["class1"]["students"].append("Daisy")
```

```
print(school["class1"]["students"])
```

Экспресс-опрос

?

Какой метод используется для удаления элемента из словаря?

Пример удаления ключа



Удаление ключа из вложенного словаря

```
del school["class2"]["teacher"]
```

```
print(school["class2"])
```

Практическое применение



Словари со вложенными структурами удобны для работы с данными, имеющими сложную иерархию.

Например



JSON-файлы



Результаты сложных вычислений



Конфигурации приложений

Копирование словарей

Поверхностное

Глубокое

Экспресс-опрос

?

Какой метод используется для поверхностного копирования словаря?



Поверхностное копирование словарей

Метод `copy()` создаёт новый словарь с копией всех ключей и значений из исходного словаря. Ключи и значения копируются "по ссылке", что означает, что изменения в вложенных объектах (например, списках) будут отражаться и в оригинале.

Пример



```
original_dict = {"name": "Alice", "age": 30, "scores": [90, 85, 88]}
copied_dict = original_dict.copy()
print(copied_dict)

# Изменение не затронет копию для неизменяемых элементов
original_dict["age"] = 31

# Изменение затронет копию для изменяемых элементов
original_dict["scores"].append(80)
print(original_dict)
print(copied_dict)
```

Экспресс-опрос

?

Какой модуль и метод используется для глубокого копирования словаря?



Глубокое копирование словарей

Глубокое копирование создаёт полностью независимую копию словаря, включая все вложенные объекты. Изменения во вложенных объектах копии не будут затрагивать оригинал и наоборот. Совершается с помощью функции `deepcopy` модуля `copy`

Использование функции `deepcopy()`:



```
import copy

original_dict = {"name": "Alice", "age": 30, "scores": [90, 85, 88]}

deep_copied_dict = copy.deepcopy(original_dict)

print(deep_copied_dict)

# Любые изменения во вложенном объекте не затронут копию

original_dict["age"] = 31

original_dict["scores"].append(80)

print(original_dict)

print(deep_copied_dict)
```



Dict comprehension

Эта структура позволяет создавать словари с использованием краткой и удобной конструкции, подобно генераторам списков. Это делает код более лаконичным и читаемым, особенно при создании или преобразовании словарей из других итерируемых объектов

Синтаксис



```
new_dict = {key_expr: value_expr for item in iterable}
```

key_expr — выражение для формирования ключа.

value_expr — выражение для формирования значения.

iterable — любой итерируемый объект (список, диапазон и т. д.).

Примеры использования dict comprehension



```
# Создание словаря с квадратами чисел из списка
numbers = [1, 2, 3, 4]
squared_dict = {num: num ** 2 for num in numbers}
print(squared_dict)
```

```
# Фильтрация элементов
original_dict = {"a": 5, "b": 2, "c": 0, "d": 3, "e": 0, "f": 3}
filtered_dict = {key: value for key, value in original_dict.items() if value > 0}
print(filtered_dict)
```

```
# Анализ данных и сохранение результатов
words = ["apple", "banana", "cherry"]
length_dict = {word: len(word) for word in words}
print(length_dict)
```


Сравнение словарей



В Python можно сравнивать словари по их содержимому, используя стандартные операторы сравнения. Сравнение словарей происходит на основании ключей и их соответствующих значений

Экспресс-опрос

?

В каком случае два словаря считаются одинаковыми?



Сравнение на равенство (==)

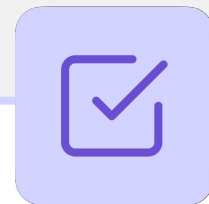
Два словаря считаются равными, если они содержат одинаковые пары "ключ-значение", независимо от порядка их добавления

Синтаксис



```
dict1 = {"a": 1, "b": 2}  
dict2 = {"b": 2, "a": 1}  
print(dict1 == dict2)  # True
```

Важно



Если хотя бы одна пара "ключ-значение" отличается, словари считаются неравными.

Синтаксис



```
dict1 = {"a": 1, "b": 2}
dict2 = {"a": 1, "b": 2, "c": 3}
print(dict1 == dict2)  # False
```

Экспресс-опрос

?

В каком случае два словаря считаются неравными?



Сравнение на неравенство (!=)

Если словари имеют разные пары "ключ-значение", они считаются неравными

Синтаксис



```
dict1 = {"a": 1, "c": 1, "b": [2, 1, 5]}
dict2 = {"b": [2, 1, 5], "a": 1, "c": 1}
print(dict1 != dict2)  # False
```

Особенности сравнения словарей



Порядок элементов не влияет на результат сравнения.



Если словари содержат изменяемые объекты (например, списки), то при сравнении учитываются их значения, а не ссылки на объекты.



Доступны только операции сравнения на равенство (==) и неравенство (!=).

Экспресс-опрос

?

Что такое функция?



Функция

Это именованный блок кода, предназначенный для выполнения определённой задачи. Функции позволяют переиспользовать код, делая программы более организованными и читаемыми.

Экспресс-опрос

?

Зачем нужны функции?

Зачем нужны функции?



Повторное использование кода: один раз написав функцию, её можно вызывать многократно



Читаемость: функции помогают разбивать программу на логические части



Модульность: удобно структурировать код, разбивая его на независимые части



Упрощение отладки: изменения в функции автоматически применяются во всех местах её вызова

Синтаксис



```
def function_name(parameters):  
    # Тело функции: код, выполняющийся при вызове  
    return result # Возвращаемое значение (опционально)
```

Пояснения

def: ключевое слово, обозначающее создание функции

function_name: имя функции, используемое для её вызова

parameters: входные данные (аргументы), которые передаются в функцию

return: возвращает результат выполнения функции (необязательно)

Пример



```
def greet(name):  
    print(f"Привет, {name}!")  
  
greet("Алиса")
```




def

Это ключевое слово, которое используется для определения пользовательской функции



Имя функции

Это уникальный идентификатор, используемый для её вызова. Оно должно быть информативным и описывать, что делает функция

Экспресс-опрос

?

Какое ключевое слово используется в качестве заглушки вместо будущего блока кода?



pass

Это ключевое слово, которое используется как заглушка. Оно позволяет определить пустой блок кода, который не выполняет никаких действий. Это полезно, когда вы планируете добавить код позже, но хотите сохранить корректный синтаксис

Особенности



Ничего не делает: `pass` ничего не выполняет, он просто позволяет избежать ошибок синтаксиса в местах, где блок кода обязателен.



Часто используется для заглушек: Позволяет временно оставить место под код, который будет добавлен позже.



Применяется в конструкциях с обязательным блоком кода: Например, в циклах, функциях, условиях.

Пример



```
#
if
    pass # Блок кода будет добавлен позже
else:
    print("False")
```

```
#
for i in
    pass # Цикл ничего не делает, но синтаксически корректен
```

```
#
def
    pass # Функция пока не реализована
```

Цикл
range(5):

Функция
validate_data():

Экспресс-опрос

?

Какие есть правила именования функций?

Правила именования функций



Имя должно начинаться с буквы или символа `_`, но не с цифры.



Нельзя использовать зарезервированные слова Python (например, `def`, `return`, `if`).



Не рекомендуется переопределять встроенные функции (`print`, `sum`).



Следуйте соглашению PEP 8: используйте `snake_case`.



Используйте глаголы и информативные имена:

Пример



```
def calculate_total(): # Хорошо  
    pass
```

```
def total(): # Плохо, не указывает на действие  
    pass
```



ВОПРОСЫ





Вызов функции

Это процесс выполнения ранее определённого блока кода (функции) с помощью её имени. При вызове можно передать функции необходимые аргументы и получить результат её работы.

Пример вызова функции без аргументов



```
def greet():  
    print("Hello!")
```

```
# Вызов функции без аргументов
```

```
greet()
```

Пример вызова функции с аргументами



```
def greet_person(name):  
    print(f"Hello, {name}!")  
  
# Вызов функции с аргументами  
greet_person("Alice")
```

Экспресс-опрос

?

Что такое аргументы функции?



Аргументы функций

Это значения, которые передаются функции при её вызове. С их помощью можно передавать данные, которые функция использует для выполнения своих задач.

Экспресс-опрос

?

Какие есть типы аргументов функций?

Типы аргументов функций

1

Позиционные
аргументы

2

Именованные
аргументы

3

Аргументы по
умолчанию

Позиционные аргументы функций



Передаются функции в порядке, указанном в определении.



Порядок важен: каждое значение будет присвоено соответствующему параметру.



Количество ожидаемых аргументов должно совпадать с количеством переданных аргументов.

Пример



```
def greet(name, age): # Ожидаемые аргументы
    print(f"My name is {name} and I am {age} years old.")
```

```
greet("Alice", 25) # Переданные аргументы
```

```
greet("Bob", 30) # Переданные аргументы
```

Значения "Alice" и 25 передаются в переменные name и age соответственно. Теперь переменные name и age можно использовать внутри функции. При каждом вызове переменные принимают новые значения.

Экспресс-опрос



Какое исключение возникает, когда операция или функция применяется к объекту неподходящего типа?



TypeError

Это исключение, которое возникает, когда операция или функция применяется к объекту неподходящего типа. Эта ошибка сигнализирует, что программа пытается выполнить действие, которое не поддерживается данным типом данных.

TypeError



Например, `TypeError` возникает если количество переданных аргументов не совпадает с количеством ожидаемых аргументов.

Пример: передано меньше аргументов, чем ожидается



```
def greet(name, age): # Ожидаемые аргументы  
    print(f"My name is {name} and I am {age} years old.")
```

```
greet("Alice") # Ошибка: меньше чем ожидается
```

Пример: передано больше аргументов, чем ожидается



```
def greet(name, age): # Ожидаемые аргументы
    print(f"My name is {name} and I am {age} years old.")

greet("Alice", 25, "Minsk") # Ошибка: больше чем ожидается
```


Именованные аргументы



Передаются с указанием имени параметра, что делает вызов функции более понятным. Порядок следования не имеет значения.

Пример



```
def greet(name, age): # Ожидаемые аргументы  
    print(f"My name is {name} and I am {age} years old.")
```

```
greet(age=30, name="Bob") # Именованные аргументы
```

Аргументы по умолчанию



При определении функции можно указать значения по умолчанию для аргументов. Если аргумент не передан, используется значение по умолчанию. Сначала указываются обязательные аргументы (без значения по умолчанию), а затем — аргументы со значениями по умолчанию. Нарушение этого порядка приведёт к ошибке синтаксиса.

Пример



```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")
```

```
greet("Alice") # Приветствие не передано, будет использовано "Hello"
```

```
greet("Bob", "Hi") # Вывод: Hi, Bob!
```

Экспресс-опрос

?

Что такое упаковка аргументов?



Упаковка аргументов

Это процесс, при котором передаваемые в функцию аргументы объединяются в одну коллекцию.

Коллекцией, в которую упакованы аргументы может быть:



кортеж для позиционных аргументов (с помощью символа `*`)



словарь для именованных аргументов (с помощью символов `**`).

Экспресс-опрос

?

Какие специальные параметры используются для упаковки аргументов?



Параметры `*args` и `**kwargs`

Это специальные параметры, которые позволяют передавать в функцию переменное количество аргументов.



***args**

Этот параметр позволяет передавать любое количество позиционных аргументов (в том числе ни одного). Аргументы упаковываются в кортеж.

Экспресс-опрос

?

Какие особенности есть у *args?

Особенности `*args`



Имя `args` не является зарезервированным и может быть заменено на любое другое (например, `*numbers` или `*values`), но стандартное соглашение — использовать `args`



Символ `*` перед параметром функции инициирует упаковку всех переданных позиционных аргументов в кортеж



Далее переменная используется без символа `*`

Синтаксис



```
def function_name(*args):  
    pass
```

#args – это кортеж, содержащий переданные аргументы

Пример



```
def calculate_sum(*args):
    print("Аргументы:", args)
    print("Сумма:", sum(args))
```

```
calculate_sum(1, 2, 3)
```

```
calculate_sum()
```

Экспресс-опрос

?

Какие особенности есть у *kwargs?



****kwargs**

Этот параметр позволяет передавать любое количество именованных аргументов. Аргументы упаковываются в словарь.

Особенности *kwargs



Имя `kwargs` также не является зарезервированным и может быть заменено на любое другое (например, `**options`), но стандартное соглашение — использовать `kwargs`



Символы `**` перед параметром функции инициируют упаковку всех переданных именованных аргументов в словарь



Далее переменная используется без символов `**`

Синтаксис



```
def function_name(*kwargs):  
    pass
```

#kwargs – это кортеж, содержащий переданные аргументы

Пример



```
def print_user_info(**kwargs):  
    print("Информация о пользователе:")  
    for key, value in kwargs.items():  
        print(f"\t{key}: {value}")  
  
print_user_info(name="Alice", age=25, city="New York")  
print_user_info()
```

Экспресс-опрос

?

Можно ли сочетать разные типы аргументов в одной функции?

Комбинация различных типов аргументов



В одной функции можно использовать разные типы аргументов. При этом важно соблюдать их порядок.

Экспресс-опрос



В каком порядке нужно использовать типы аргументов?

Порядок использования аргументов



Позиционные аргументы



`*args`



Аргументы по умолчанию



`**kwargs`

Пример



```
def show_full_info(name, *args, age=25, **kwargs):
```

```
    print(f"Name: {name}")
```

```
    print(f"Other details: {args}")
```

```
    print(f"Age: {age}")
```

```
    print(f"Additional info: {kwargs}")
```

```
show_full_info("Alice", "Developer", age=30, city="New York", hobby="Reading")
```


Экспресс-опрос

?

Какое ключевое слово используется для возвращения из функции в место ее вызова?



return

Это ключевое слово, которое используется для возврата значения из функции в место её вызова. Оно завершает выполнение функции и возвращает указанное значение (или **None**, если значение не указано)

Особенности return



Возврат значения: `return` позволяет передать результат работы функции обратно к вызывающему коду.



Завершение функции: После выполнения инструкции `return` функция завершает свою работу, даже если после неё есть другие строки кода.



Отсутствие значения: Если `return` вызывается без указания значения, функция возвращает `None`.



Отсутствие return: Даже если `return` отсутствует или не достигнут, функция выполняет код до конца и возвращает `None`.



Несколько return: Функция может содержать несколько `return`. В этом случае выполнится первый достигнутый `return`.

Синтаксис



```
def function_name(parameters):  
    # тело функции  
    return value
```

Возврат значения



Пояснение

Функция `add` возвращает сумму чисел, которая сохраняется в переменной `result`.

Код

```
def add(a, b):  
    return a + b  
  
result = add(3, 5)  
print(result)
```

Возврат одного из значений



Пояснение

Если условие выполняется, функция завершается после первого `return`, и код после него не исполняется.

Код

```
def check_positive(number):
    if number > 0:
        return "Положительное число"
    return "Отрицательное или ноль"

print(check_positive(10))
print(check_positive(-5))
```

Возврат None



Пояснение

Функция `say_hello` ничего не возвращает, поэтому её результат равен `None`.

Код

```
def say_hello():
    print("Hello, World!")

result = say_hello()

print(result)
```

Множественный возврат значений



Пояснение

`return` может возвращать несколько значений, которые упаковываются в кортеж.

Код

```
def calculate(a, b):  
    return a + b, a - b  
  
result = calculate(10, 5)  
print(result)
```


Пустой return



calculate_factorial(n):

```
def
    if n < 0:
        return # Завершаем функцию без вычислений

    result = 1

    for i in range(1, n + 1):
        result *= i

    return result
```

```
num1 = -5
print(f"Факториал числа {num1}: {calculate_factorial(num1)}")
```

```
num2 = 5
print(f"Факториал числа {num2}: {calculate_factorial(num2)}")
```



ВОПРОСЫ



Экспресс-опрос

?

Что такое область видимости?



Область видимости

Это контекст (границы), в котором переменные доступны для использования. Она определяет, где можно обращаться к данным и какие переменные доступны в разных частях программы.

Local (Локальная область)



Область действия: ограничена функцией, в которой переменная была объявлена.



Срок жизни: существует только во время выполнения функции.



Определение: локальная переменная создаётся внутри функции и доступна только в её пределах.

Пример



```
def my_function():  
    local_var = 10 # Локальная переменная  
    print(f"Локальная переменная: {local_var}")  
  
my_function()  
  
# print(local_var) # Ошибка: local_var недоступна за пределами функции
```

Global (Глобальная область)



Область действия: доступна во всей программе, включая функции.



Срок жизни: существует, пока выполняется программа.



Определение: объявляется вне функций или других блоков и доступна во всей программе.

Пример



```
global_var = 20 # Глобальная переменная

def show_global():
    print(f"Глобальная переменная: {global_var}")

show_global()

print(global_var) # Глобальная переменная доступна в любом месте
```


Built-in (Встроенные объекты)



Эта область содержит встроенные функции и объекты Python (например, `len()`, `print()`, `int()`).



Они доступны в любом месте программы, если не переопределены.

Пример



```
print(len("Hello")) # Вызов встроенных функций len и print
```

Экспресс-опрос

?

В каком порядке Python ищет переменные?

Python ищет переменные



Local — в локальной области функции.



Enclosing — в области окружающих функций (замыкания).



Global — среди глобальных переменных.



Built-in — среди встроенных объектов.

Примеры работы LEGB



Пояснение

Если переменная отсутствует во всех четырёх областях, **Python выбрасывает исключение `NameError`**.

Код

```
# x = 10      # Глобальная переменная
def function():
    # x = 5    # Локальная переменная
    print(x)

function()
```

Примеры работы LEGB



Пояснение

Если внутри функции объявляется переменная с тем же именем, что и глобальная, **локальная переменная перекрывает доступ к глобальной**.

Код

```
x = 10      # Глобальная переменная
def my_function():
    x = 5    # Локальная переменная с тем же
            # именем
    print(f"Локальная переменная: {x}")

my_function()
print(f"Глобальная переменная: {x}")    #
Глобальная переменная остаётся неизменной
```



ВОПРОСЫ



Экспресс-опрос

?

Для чего используется ключевое слово `global`?



global

Это ключевое слово в Python, которое позволяет изменять значение глобальной переменной внутри функции

Использование global



Пояснение

variable_name — имя глобальной переменной, к которой требуется доступ.

Синтаксис

```
global variable_name
```

Пример без global



```
count = 0 # Глобальная переменная
```

```
def increment_counter():
```

```
    count = count + 1 # Ошибка, так как `count` внутри функции считается локальной
```

```
    print(count)
```

```
increment_counter()
```

Код вызовет **UnboundLocalError**, так как Python воспринимает `count` как локальную переменную, но мы пытаемся использовать её до присваивания.

Пример с global



```
count = 0 # Глобальная переменная
```

```
def increment_counter():
```

```
    global count # Указываем, что работаем с глобальной переменной
```

```
    count += 1
```

```
increment_counter()
```

```
print(count)
```

```
increment_counter()
```

```
print(count)
```

Экспресс-опрос

?

В чем состоит опасность чрезмерного использования ключевого слова `global`?

Особенности использования `global`



Изменяет глобальную переменную, а не создаёт новую локальную.



Не требуется при **чтении** глобальной переменной внутри функции, но нужен при **изменении**.



Чрезмерное использование `global` может привести к трудноотслеживаемым ошибкам, поэтому его следует применять с осторожностью.

Зачем передавать аргументы в функцию



В программировании рекомендуется передавать значения в функцию через параметры, а не использовать глобальные переменные внутри функции. Это связано с несколькими важными принципами и проблемами, которые могут возникнуть при работе с глобальными переменными.

Экспресс-опрос

?

Почему лучше передавать аргументы в функцию, а не использовать глобальные переменные?

Зачем передавать аргументы в функцию



Повышение читаемости и предсказуемости кода: Когда функция получает значения через параметры, она становится независимой от внешних переменных.



Предотвращение неожиданных ошибок: Использование глобальных переменных внутри функций может привести к неожиданным результатам.



Упрощение тестирования и переиспользования кода: Функции, которые зависят от глобальных переменных, трудно тестировать.



Избегание конфликтов с именами переменных: Глобальные переменные могут случайно перезаписаться в разных частях программы.

Пример с аргументами



Все значения передаются в функцию явно, что делает код понятным

```
def calculate_area(width, height):
```

```
    return width * height
```

```
result = calculate_area(5, 10)
```

```
print(result)
```

Пример с глобальными переменными



```
width = 5
```

```
height = 10
```

```
def calculate_area():
```

```
    # Непонятно, откуда берутся width и height, если смотреть только на функцию
```

```
    return width * height # Использует глобальные переменные
```

```
result = calculate_area()
```

```
print(result)
```

Когда НЕ стоит использовать global



Для передачи данных между функциями — лучше передавать параметры.



В больших программах — сложно отслеживать, где изменяется переменная.



Для временных значений — лучше использовать локальные переменные.

Экспресс-опрос

?

В каких случаях использование глобальных переменных оправдано?

Когда global все же оправдан



Для изменения переменных на уровне модуля. Например, в настройках программы для хранения конфигураций.



В небольших скриптах, где глобальные переменные помогают быстро решить задачу и масштабируемость не важна.



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов.



Задание 1

Числа кратные 3 или 5

Напишите программу, которая создает множество из чисел, фильтруя элементы, не кратные 3 или 5.

Данные

```
numbers = [16, 18, 1, 6, 3, 2, 6, 2, 14, 3, 20, 15, 19, 4, 18, 15, 15, 4, 20, 17]
```

Пример вывода

```
{3, 6, 15, 18, 20}
```

Задание 2

Проверка уникальности ключей

Напишите программу, которая проверяет, содержатся ли в двух заданных словарях одинаковые ключи. Вывести одинаковые ключи или "-", если таковых нет.

Данные

```
dict1 = {"a": 1, "b": 2, "c": 3}
dict2 = {"b": 5, "d": 7, "a": 8}
```

Пример вывода

Общие ключи: ['a', 'b']

Задание 3

Строки с длиной

Напишите программу, которая преобразует список строк в словарь, где ключи — сами строки, а значения — квадраты их длины.

Данные

```
words = ["apple", "banana", "cherry", "date"]
```

Пример вывода

```
{'apple': 25, 'banana': 36, 'cherry': 36, 'date': 16}
```

Задание 4

Проверка подмножества

Напишите программу, которая проверяет, является ли один из словарей подмножеством другого (т.е. все пары "ключ-значение" одного словаря содержатся в другом словаре).

Данные

```
dict1 = {"a": 1, "b": 2}  
dict2 = {"a": 1, "b": 2, "c": 3}
```

Пример вывода

Первый словарь является подмножеством второго.

Задание 5

Удаление пустых значений

Напишите программу, которая удаляет из словаря все пары "ключ-значение", где значение пустое (например, None, пустая строка или пустой список).

Данные

```
data = {"a": None, "b": 2, "c": "", "d": [], "e": [1, 2]}
```

Пример вывода

```
{'b': 2, 'e': [1, 2]}
```

Задание 6

Потерянные страницы книги

Вам дан словарь, где ключи — номера страниц книги, а значения — содержимое страниц. Некоторые страницы отсутствуют (значения None). Напишите программу, которая на пропущенных страницах заменит значение на "Страница потеряна".

Данные

```
book = {1: "Начало истории", 2: None, 3: "Глава 1", 4: None, 5: "Глава 2"}
```

Пример вывода

Изменённый словарь: {1: 'Начало истории', 2: 'Страница потеряна', 3: 'Глава 1', 4: 'Страница потеряна', 5: 'Глава 2'}

Задание 7

База оценок студентов

У вас есть словарь с именами студентов и списками их оценок. Напишите программу, которая вычисляет средний балл для каждого студента. Далее нужно сохранить средний балл в значениях для каждого студента, как показано на примере.

Данные

```
grades = {
    "anna": [5, 4, 3, 5],
    "bennet": [3, 2, 4],
    "john": [5, 5, 5]
}
```

Пример вывода

```
{'anna': {'оценки': [5, 4, 3, 5], 'средний балл': 4.25}, 'bennet': {'оценки': [3, 2, 4],
'средний балл': 3.0}, 'john': {'оценки': [5, 5, 5], 'средний балл': 5.0}}
```


Задание 8

Одинаковые предметы

Есть список студентов и наборы предметов, которые они изучают. Необходимо сгруппировать студентов по идентичным наборам предметов, используя `frozenset` как ключ, и вывести группы.

Данные

```
students = {
    "Alice": ["Math", "Physics"],
    "Bob": ["Math", "Physics"],
    "Charlie": ["Chemistry", "Biology"],
    "David": ["Math", "Physics"],
    "Eve": ["Chemistry", "Biology"]
}
```

Пример вывода

```
Группа с предметами: Physics,
Math: ['Alice', 'Bob', 'David']
```

```
Группа с предметами: Biology,
Chemistry: ['Charlie', 'Eve']
```

Задание 9

Перевод и расширение словаря

Напишите программу, которая позволяет пользователю вводить английские слова и получать их перевод на русский из словаря. Если слово отсутствует в словаре, программа должна выводить сообщение "Перевод отсутствует".

В конце программа должна предложить пользователю добавить новые слова в словарь.

Данные

```
english_words = {  
    "Butterfly": "Бабочка",  
    "Training": "Обучение",  
    "Restaurant": "Ресторан",  
    "Programming": "Программирование",  
}
```

Задание 9

Перевод и расширение словаря

Данные

```
english_words = {
    "Butterfly": "Бабочка",
    "Training": "Обучение",
    "Restaurant": "Ресторан",
    "Programming": "Программирование",
}
```

Пример вывода

Введите слово на английском (или 'exit' для выхода):

Butterfly

Перевод: Бабочка

Введите слово на английском (или 'exit' для выхода):

Travel

Перевод отсутствует.

Хотите добавить перевод? (да/нет): да

Введите перевод для слова "Travel": Путешествие

Слово "Travel" добавлено в словарь.

Введите слово на английском (или 'exit' для выхода):

exit

Программа завершена.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Распределение студентов по группам

У вас есть словарь, где ключи — имена студентов, а значения — их баллы за экзамен. Необходимо распределить студентов по группам:

- **"Отличники"**: баллы ≥ 85
- **"Хорошисты"**: баллы от 70 до 84
- **"Троечники"**: баллы от 50 до 69
- **"Не сдали"**: баллы < 50

Создайте словарь с ключами-группами и словарями со студентами в качестве значений.

Данные

```
students = {"Аня": 92, "Боря": 76, "Ваня": 65, "Галя": 48, "Дима": 88, "Ева": 54}  
groups = ["Отличники", "Хорошисты", "Троечники", "Не сдали"]
```

Домашнее задание

Реверс словаря

Напишите программу, которая меняет местами ключи и значения в словаре. Если значения повторяются, добавьте их в список.

Данные

```
data = {"a": 1, "b": 2, "c": 1, "d": 3}
```

Пример вывода

Перевернутый словарь: {1: ['a', 'c'], 2: ['b'], 3: ['d']}

Домашнее задание

Простое число

Напишите функцию, которая проверяет, является ли число n простым (делится только на 1 и само себя) и возвращает булевый результат.

Данные:

$n = 17$

Пример вывода:

Число 17 является простым

Домашнее задание

Фильтрация чисел по чётности

Напишите функцию, которая принимает filter_type ("even" или "odd") и произвольное количество чисел, возвращая только те, которые соответствуют фильтру.

Пример вызова:

```
print(filter_numbers("even", 1, 2, 3, 4, 5, 6))  
print(filter_numbers("odd", 10, 15, 20, 25))  
print(filter_numbers("prime", 2, 3, 5, 7))
```

Пример вывода:

[2, 4, 6]

[15, 25]

Некорректный фильтр

Домашнее задание

Объединение словарей

Напишите функцию, которая принимает любое количество словарей и объединяет их в один. Если ключи повторяются, используется значение из последнего словаря.

Данные:

```
dict1 = {"a": 1, "b": 2}  
dict2 = {"b": 3, "c": 4}  
dict3 = {"d": 5}
```

Пример вызова:

```
print(merge_dicts(dict1, dict2, dict3))
```



ВОПРОСЫ



Заключение

